

Feature Activation — Bit Signalling

How the network upgrades its own rules without a flag-day — miners vote by setting bits in blocks, and a feature activates when enough support accumulates over a schedule.

SUBJECT hathor-core · Part II · the node, end to end

CHAPTER 38 · Part II · The Node

BRANCH study/hathor-core-studies

EDITION 2026 · v1

Feature activation · Bit signalling · BIP9-style · Feature state machine · Evaluation window · Threshold · signal_bits · FeatureService · Miner voting

Table of Contents

Chapter 38 — Feature Activation: Bit Signalling	3
38.1 Localization	3
38.2 What it does and why it exists	4
The problem: you cannot push an update to a network you don't own	4
The shape of the answer: schedule it, and gate it on demonstrated support	5
38.3 The concepts it rests on	5
Two new concepts unique to this chapter	6
38.4 The code, walked	7
38.4.1 The Feature enum — the catalogue of upgrades	7
38.4.2 Criteria — one feature's schedule	7
38.4.3 The six states	9
38.4.4 The state machine	9
38.4.5 Counting the votes — feature_activation_bit_counts	11
38.4.6 FeatureService — the read side (the brain)	12
38.4.7 Gating behaviour — two real use-sites	13
38.4.8 BitSignalingService — the write side (this node's ballot)	14
38.4.9 A small worked example	15
38.5 How it plugs into the lifecycle	16
38.6 Recap	17

Chapter 38 — Feature Activation: Bit Signalling

What you'll learn in this chapter

- Why a decentralized network cannot upgrade its rules by "just pushing an update," and what a fork is — the failure this whole subsystem exists to prevent.
- The idea of **activation by vote over a schedule**: miners advertise support for a proposed rule change by setting bits in the blocks they produce, and the change switches on only once enough support has accumulated.
- The **feature state machine** — `DEFINED` → `STARTED` → `(MUST_SIGNAL | LOCKED_IN)` → `ACTIVE` | `FAILED` — its six states and exactly what makes a feature move from one to the next.
- How a vote is physically carried: the `signal_bits` byte on a block, and the rolling `feature_activation_bit_counts` the node keeps so it can ask "how many of the last N blocks voted yes?".
- How the rest of the node asks **"is feature X active here?"** (`FeatureService.is_feature_active`) and changes its behaviour on the answer — with one real verification example.

A blockchain's rules are not frozen forever. New transaction types get added, a script opcode gets a tighter bound, nano-contracts get switched on. Each of these is a change to what counts as a valid block or transaction. The hard part is not writing the new rule — it is turning it on everywhere at once, on a network of thousands of independent computers that no one controls, where you cannot phone each operator and tell them to restart at noon. This chapter is about the machinery Hathi uses to coordinate that turn-on safely. It is small in code (under a thousand lines, all in `hathi/feature_activation/`) but it sits on top of a genuinely subtle idea, so most of this chapter is spent making that idea concrete.

38.1 Localization

The whole subsystem lives in one package, with one helper reaching into the vertex model.

```

hathor-core/
├── hathor/
│   ├── feature_activation/
│   │   ├── feature.py
│   │   ├── settings.py
│   │   ├── feature_service.py
│   │   ├── bit_signaling_service.py
│   │   ├── utils.py
│   │   └── model/
│   │       ├── criteria.py
│   │       ├── feature_state.py
│   │       └── feature_info.py
│   ├── storage/
│   │   └── feature_activation_storage.py
│   └── resources/feature.py
├── transaction/
│   ├── base_transaction.py
│   ├── block.py
│   └── static_metadata.py

```

◀ YOU ARE HERE

- ← the Feature enum (the catalogue of upgrades)
- ← Settings: evaluation_interval, threshold, features{}
- ← FeatureService: get_state + is_feature_active (the l
- ← BitSignalingService: produces THIS node's votes
- ← Features dataclass (a convenience snapshot)
- ← Criteria: one feature's schedule (bit, heights, thr
- ← FeatureState enum (the 6 states)
- ← FeatureInfo = (criteria, state) pair
- ← persists/validates the settings used
- ← HTTP endpoint exposing feature states
- ← signal_bits field lives here (base_transaction.py:1
- ← reads signal_bits → bit list (block.py:307)
- ← feature_activation_bit_counts (static_metadata.py:7

Where this sits in the node (recap). Feature activation is one of the higher services from the module map (Ch. 0, §0.4) — it is built on top of the base ledger, not part of it. Two of its services are wired into the `HathorManager` at build time: `FeatureService` (the read side — answers questions) and `BitSignalingService` (the write side — produces this node's votes). It plugs into **verification** (Ch. 31), which gates rules on it, and into **mining** (Ch. 37), which stamps the vote onto each new block template.

38.2 What it does and why it exists

The problem: you cannot push an update to a network you don't own

Think about how a normal app ships a new feature. The company controls the servers; it deploys the new code, and from that instant every client talks to the new version. There is one authority, and it flips the switch.

A blockchain has no such authority — that is the entire point of it (Ch. 6). The "servers" are thousands of independent full nodes, run by people who don't know each other, who upgrade their software whenever they feel like it, if at all. Some run the latest release the day it drops; some run a version from two years ago; some never upgrade. There is no one who can flip a switch for everyone.

Now suppose a new release tightens a rule — say, "from now on, a transaction may not use opcode X." Imagine the developers ship it and it takes effect immediately on upgrade. Watch what happens:

- A miner running the **new** code produces a block that obeys the new rule. Fine.

- A miner running the **old** code produces a block that uses opcode X. Under the old rules, that block is perfectly valid.
- The new node receives the old node's block and **rejects** it as invalid. The old node receives the new node's blocks and accepts them. The two nodes now disagree about which blocks belong in the chain.

From this moment the network has split into two: one set of nodes building on a history the other set refuses. This split is called a **fork**¹. A fork is not a cosmetic bug — it means two groups of users now have two different, incompatible ledgers. A coin spent on one side was never spent on the other. The shared truth that made the system worth anything is gone.

So the question feature activation answers is precise: **how do you change a consensus rule so that essentially every node enforces the new rule starting at the same block, even though the nodes upgraded their software at wildly different times?**

The shape of the answer: schedule it, and gate it on demonstrated support

You cannot make everyone upgrade at the same wall-clock moment. But you can make the new rule take effect at the same block height for everyone — height is a property of the shared ledger, so all nodes agree on it. That turns "coordinate in time" into "coordinate in the data," which is solvable.

But picking a height up front is risky on its own. If you hard-code "rule X activates at height 5,000,000" and it turns out most of the network's mining power hasn't upgraded by then, you have scheduled a fork — at height 5,000,000, the un-upgraded majority keeps producing blocks the upgraded minority rejects. So the design adds a second guard: **before the rule turns on, the network must demonstrate that enough of it is ready**. "Enough of it is ready" is measured by miners voting. Each miner that has upgraded sets a bit in the blocks it produces, signalling "I support this change and I'm ready to enforce it." The rule activates only once that vote crosses a threshold inside a defined window. If the vote never gets there, the proposal fails cleanly instead of forking the chain.

This is a vote-to-activate-over-a-window scheme. It is modelled directly on Bitcoin's **BIP9**², which solved exactly this problem first. The rest of the chapter makes each piece — the schedule, the window, the threshold, the vote, and the state machine that ties them together — concrete.

38.3 The concepts it rests on

Three ideas from earlier chapters carry this one. Quick recaps, then the new material.

Miners, blocks, and `signal_bits` (full treatment in Ch. 9 and Ch. 25). A miner is a participant that produces new **blocks** by solving a proof-of-work puzzle (Ch. 9). Every block carries a small `signal_bits` field — one byte — alongside its other contents (`base_transaction.py:168`). Feature activation borrows the low 4 bits of that byte as a ballot: each bit is one feature's "yes/no" checkbox, and a miner fills them in on every block it mines. The block is the ballot; the miner is the voter.

The settings schedule (full treatment in Ch. 22). Constants that define the network — genesis, timing, and the feature schedule — live in the settings profile selected at boot. The feature-activation settings are a sub-block of that (`settings.FEATURE_ACTIVATION`), listing every feature and its activation criteria. Because the schedule is part of settings, every node on the same network shares it exactly; that shared schedule is what lets independent nodes reach the same conclusion about when a feature turns on.

A state machine (recall Ch. 34's footnote). A state machine is a model where a thing is always in exactly one of a fixed set of named **states**, and moves between them only along defined **transitions**, each guarded by a condition. We met one for peer connections in Ch. 34 (`HELLO → PEER_ID → READY`). Feature activation has its own: each feature, at each block, is in one of six states, and the transitions are evaluated once per scheduling window. This is the core of the subsystem.

Two new concepts unique to this chapter

The evaluation interval (the window). Counting votes block-by-block would be noisy and would let a feature flicker on and off. Instead the schedule is divided into fixed-length **windows** called evaluation intervals. On Hathor's mainnet an interval is 20,160 blocks — about one week at one block every 30 seconds (`settings.py:31`). A feature's state is recomputed **only at the boundary between windows**, using the votes gathered during the window that just ended. Inside a window the state is frozen. So a feature does not change state mid-week; it changes (if at all) at the start of each new week, based on how the previous week voted. A block whose height is an exact multiple of the interval is a **boundary block** — the only kind for which a fresh state is calculated.

The threshold. A feature activates only if the number of "yes" votes in a window reaches a **threshold** — a minimum count of blocks-in-the-window that signalled the bit. Hathor's default is 18,144 out of 20,160, which is 90% (`settings.py:38`). A high threshold is deliberate: it means a feature turns on only with the support of an overwhelming majority of recent mining power, which is precisely the "is the network ready?" guard from §38.2.

38.4 The code, walked

We build the picture bottom-up: the catalogue of features, the per-feature schedule, the six states, the state machine that moves between them, how votes are physically counted, and finally the two services the rest of the node talks to.

38.4.1 The `Feature` enum — the catalogue of upgrades

Every upgrade that has ever been (or will be) put through this process has a name in one enum.

```
@unique
class Feature(StrEnum):
    # These NOP features are used in tests
    NOP_FEATURE_1 = 'NOP_FEATURE_1'
    NOP_FEATURE_2 = 'NOP_FEATURE_2'
    NOP_FEATURE_3 = 'NOP_FEATURE_3'

    INCREASE_MAX_MERKLE_PATH_LENGTH = 'INCREASE_MAX_MERKLE_PATH_LENGTH'
    COUNT_CHECKDATASIG_OP = 'COUNT_CHECKDATASIG_OP'
    NANO_CONTRACTS = 'NANO_CONTRACTS'
    FEE_TOKENS = 'FEE_TOKENS'
    OPCODES_V2 = 'OPCODES_V2'
```

`feature.py:18`

The docstring states a discipline worth pausing on (`feature.py:20`): features are never removed from this enum, and their string values are never changed. The reason is that the values are how the settings profile maps a schedule onto a feature, and they are baked into the historical record of how past activations went. Delete or rename one and you would lose the ability to reproduce history. So the enum only grows. The three `NOP_FEATURE_*` entries (`feature.py:27`) are deliberate no-ops used by the test suite to exercise the machinery without changing any real rule.

Note the base class is `StrEnum`³ — each member is its string value, which is why `feature.value` and the member compare equal to the plain string. That matters because the settings file refers to features by their string names.

38.4.2 `Criteria` — one feature's schedule

Each feature in the schedule carries a `Criteria` object: the complete recipe for when and how that feature activates. It is a Pydantic model (Ch. 18 — validated, read-only data).

```

class Criteria(BaseModel):
    bit: NonNegativeInt
    start_height: NonNegativeInt
    timeout_height: NonNegativeInt
    threshold: Optional[NonNegativeInt] = None
    minimum_activation_height: NonNegativeInt = 0
    lock_in_on_timeout: bool = False
    version: str = Field(..., pattern=version.BUILD_VERSION_REGEX)
    signal_support_by_default: bool = False

```

`criteria.py:26` (fields at `criteria.py:59 - 66`; docstring `criteria.py:32`)

Read each field as a knob on the schedule:

- **bit** — which of the (up to four) signalling bits this feature uses. Bit 0, 1, 2, or 3.
- **start_height** — the feature does nothing until the chain reaches this height; at the first boundary at-or-after it, voting opens.
- **timeout_height** — the deadline. If the feature hasn't gathered enough votes by here, it fails (unless `lock_in_on_timeout` says otherwise).
- **threshold** — how many "yes" votes per window are needed. If left `None`, the network's `default_threshold` (90%) is used (`criteria.py:83`, `get_threshold`).
- **minimum_activation_height** — even after a feature is locked in, it won't go live before this height. A floor that gives operators warning time.
- **lock_in_on_timeout** — if `True`, reaching the timeout forces activation instead of failing. This is how a change the developers consider mandatory is shipped: it still uses the schedule, but a stalling minority cannot veto it forever (see `MUST_SIGNAL` below).
- **signal_support_by_default** — whether this node, when it mines, votes yes for this feature unless told otherwise.

There is a sibling class `ValidatedCriteria` (`criteria.py:88`) whose job is to run cross-field sanity checks once the network-wide constants are known — for example, `bit` must be below `max_signal_bits` (`criteria.py:98`), the timeout must sit at least two evaluation intervals after the start (`criteria.py:105`), and `start_height`, `timeout_height`, and `minimum_activation_height` must all be exact multiples of the evaluation interval (`criteria.py:118`). That last rule is what guarantees those heights always land on window boundaries — the only places the state machine ever recomputes.

The network-wide settings that hold all this sit in `Settings` (`settings.py:25`): `evaluation_interval` (`settings.py:31`), `max_signal_bits` (`settings.py:34`, capped at 8 — only 4 used), `default_threshold` (`settings.py:38`), and `features: dict[Feature, Criteria]` (`settings.py:43`). One validator (`settings.py:66`) refuses to let two features

share a bit during overlapping height ranges — a bit can be recycled for a later feature, but only after the earlier one's window has closed, so no block's single bit is ever ambiguous about which feature it votes for.

38.4.3 The six states

A feature is never a plain "on" or "off." At every block it is in one of six states, defined in `FeatureState` (`feature_state.py:19`):

```
class FeatureState(str, Enum):
    DEFINED = 'DEFINED'
    STARTED = 'STARTED'
    MUST_SIGNAL = 'MUST_SIGNAL'
    LOCKED_IN = 'LOCKED_IN'
    ACTIVE = 'ACTIVE'
    FAILED = 'FAILED'
```

`feature_state.py:32 - 37`

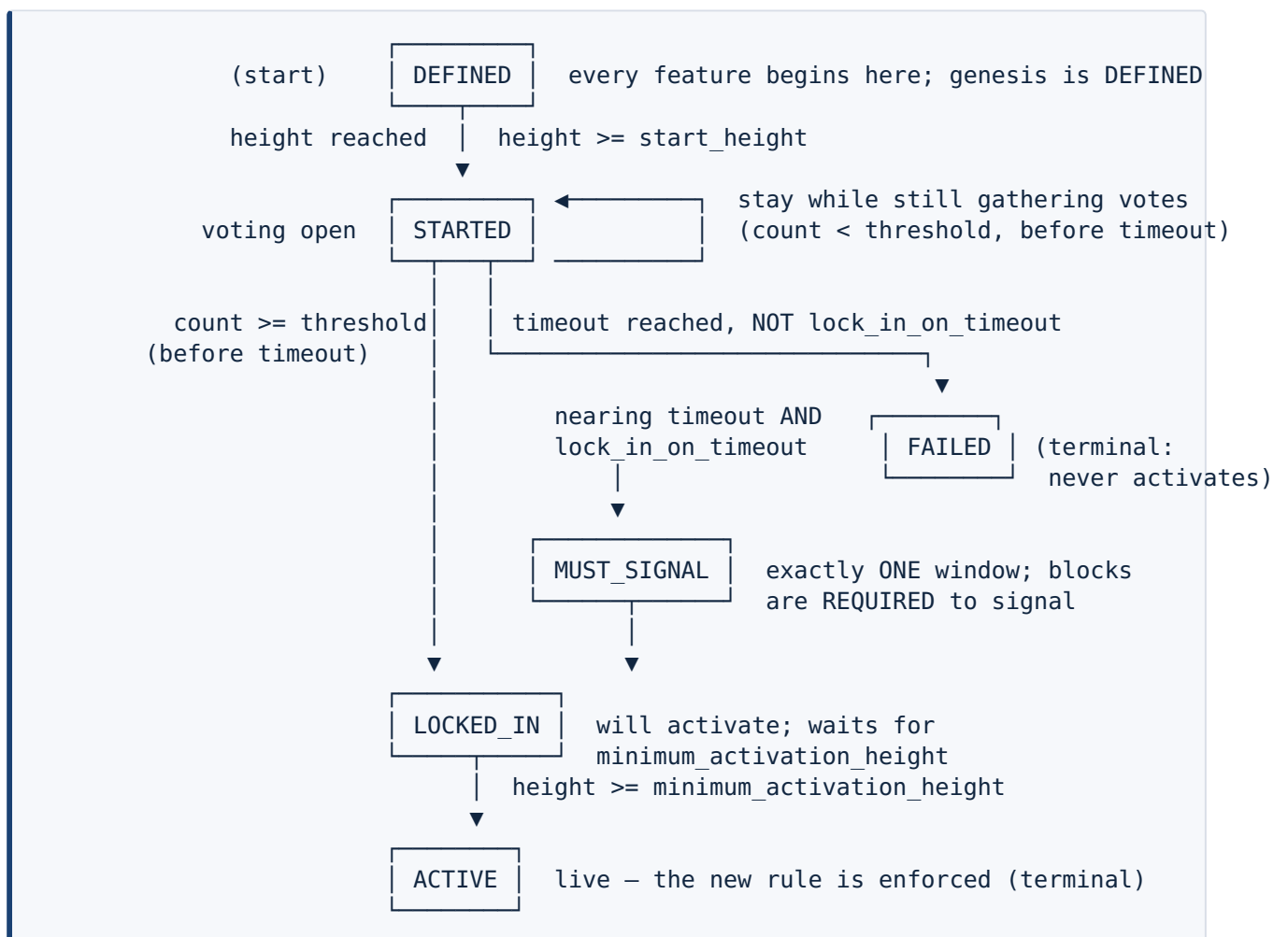
In words (docstring at `feature_state.py:20`):

- **DEFINED** — the feature exists in the catalogue but its start height hasn't been reached. Nothing is happening. Every feature starts here; genesis is **DEFINED** for everything (`feature_service.py:100`).
- **STARTED** — voting is open. Miners may signal support, and the node tallies votes each window.
- **MUST_SIGNAL** — a mandatory-feature special case: the feature is about to lock in, and the protocol now requires blocks to signal it. We cover why below.
- **LOCKED_IN** — enough support was reached. The feature will activate; this state is the grace period before it does.
- **ACTIVE** — live. The new rule is now enforced. This is the only state for which `is_active()` returns `True` (`feature_state.py:39`).
- **FAILED** — the deadline passed without enough support. The feature will never activate (on this network's history). A terminal state.

Three of these — **STARTED**, **MUST_SIGNAL**, **LOCKED_IN** — are the signalling states (`feature_state.py:43`, `get_signaling_states`): the states during which a block carrying that feature's bit is meaningful. Outside them, setting the bit has no effect.

38.4.4 The state machine

Here is the whole machine. Each arrow is evaluated **only at a window boundary**, using the criteria and the vote count from the window that just ended.



The single function that computes every one of those arrows is `_calculate_new_state` (`feature_service.py:139`). It is a tidy `if`-ladder over the previous state. The interesting transitions:

DEFINED → STARTED when the boundary height reaches `start_height` (`feature_service.py:163 - 167`). Before that, it stays **DEFINED** .

STARTED → (the fork in the road) (`feature_service.py:169 - 186`). At each boundary the node reads the vote count for the window that ended and compares it to the threshold:

```

if previous_state == FeatureState.STARTED:
    if height >= criteria.timeout_height and not criteria.lock_in_on_timeout:
        return FeatureState.FAILED

    parent_block = boundary_block.get_block_parent()
    counts = parent_block.static_metadata.feature_activation_bit_counts
    count = counts[criteria.bit]
    threshold = criteria.get_threshold(self._feature_settings)

    if height < criteria.timeout_height and count >= threshold:
        return FeatureState.LOCKED_IN

    if (height + evaluation_interval >= criteria.timeout_height) and criteria.lock_in_on_timeout:
        return FeatureState.MUST_SIGNAL

    return FeatureState.STARTED

```

feature_service.py:169

Read the four outcomes: 1. Past the timeout and not a forced feature → **FAILED**. Voting is over and it lost. 2. Votes reached the threshold while still in time → **LOCKED_IN**. It won. 3. The next window would cross the timeout and this is a forced feature (`lock_in_on_timeout`) → **MUST_SIGNAL**. Last chance to signal, and signalling is now compulsory. 4. None of the above → stay **STARTED** for another window.

MUST_SIGNAL → **LOCKED_IN**, always, after exactly one window (`feature_service.py:188 - 192`). The state exists for a single interval and then locks in unconditionally. Its purpose is to give a forced feature a final, enforced signalling window so that the historical record shows the network was told — blocks that fail to signal during this phase are rejected (we'll see the enforcement in §38.4.7).

LOCKED_IN → **ACTIVE** once the height reaches `minimum_activation_height` (`feature_service.py:194 - 198`); otherwise it waits in **LOCKED_IN**. **ACTIVE** and **FAILED** are sinks — once there, the feature stays (`feature_service.py:200 - 204`).

38.4.5 Counting the votes — `feature_activation_bit_counts`

The state machine asks "how many of the blocks in the last window set this bit?" Computing that by re-scanning thousands of blocks at every boundary would be slow. Instead the node keeps a **running tally** on each block, carried forward block by block.

That tally is `feature_activation_bit_counts` — a field on the block's static metadata⁴ (`static_metadata.py:72`). It is a small list: one integer per signalling bit, each integer being "how many blocks from the start of the current window up to and including this block have set that bit."

It is computed once, lazily, when the block's static metadata is built (`static_metadata.py:132`, `_calculate_feature_activation_bit_counts`). The logic is a one-line recurrence:

```
previous_counts = cls._get_previous_feature_activation_bit_counts(block, height, settings, vertex)
bit_list = block._get_feature_activation_bit_list()

count_and_bit_pairs = zip_longest(previous_counts, bit_list, fillvalue=0)
updated_counts = starmap(add, count_and_bit_pairs)
return list(updated_counts)
```

static_metadata.py:146 – 151

In words: take the parent's counts, add this block's own vote (a list of 0/1 per bit), element-wise. The "previous counts" come from the parent block — **unless this block is itself a boundary block**, in which case the tally resets to empty and starts fresh for the new window (`static_metadata.py:153`, `_get_previous_feature_activation_bit_counts`; the reset at `:167 – 168`). So the count climbs across a window and zeroes at each boundary. Each block's stored count is the cumulative tally for its own window so far; the count on the last block before a boundary is therefore the full window's tally — which is exactly the number `_calculate_new_state` reads from the parent block at the boundary (`feature_service.py:175`).

Where does a single block's own vote come from? From its `signal_bits` byte. `Block._get_feature_activation_bit_list` (`block.py:307`) masks off the low `max_signal_bits` bits of `signal_bits` and expands them into a list of 0s and 1s (`block.py:314 – 317`). One subtlety the code is careful about, stated in the comments: **the least-significant bit is on the left** of the list, so `bit_list[criteria.bit]` lines up with `bit` being the bit index. That is why the state machine can index the count list directly by `criteria.bit`.

38.4.6 FeatureService — the read side (the brain)

`FeatureService` (`feature_service.py:46`) is the object the rest of the node consults. It holds the feature settings and the transaction storage (`feature_service.py:49`). Its headline method is the question everyone actually wants to ask:

```
def is_feature_active(self, *, vertex: Vertex, feature: Feature) -> bool:
    """Return whether a Feature is active for a certain vertex."""
    block = self._get_feature_activation_block(vertex)
    state = self.get_state(block=block, feature=feature)
    return state.is_active()
```

feature_service.py:54

Two things to notice. First, activation is relative to a vertex. "Is feature X active?" has no absolute answer — it depends on where in the chain you ask, because the answer changes at activation height. So the query takes the vertex it is being asked about. For a block, that's the block itself; for a transaction, it resolves to the transaction's `closest_ancestor_block` (`feature_service.py:60`, `_get_feature_activation_block`) — the

deepest block the transaction depends on, recorded in its static metadata (`static_metadata.py:181`). That makes a transaction's view of "what's active" deterministic and tied to the ledger, not to wall-clock time.

Second, the real work is in `get_state` (`feature_service.py:96`), and it leans hard on the window structure to stay cheap. Its logic:

1. Genesis is `DEFINED` for everything — short-circuit (`feature_service.py:100`).
2. If this block already has the state cached in its metadata, return it (`feature_service.py:103` ; cache read via `block.get_feature_state` , `block.py:327`).
3. Otherwise, find the **previous boundary block** (`feature_service.py:109 – 114`). Every block in a window shares that window's state, so the state is really a property of the boundary. Recurse to get the boundary's state, and cache it on the boundary block (`feature_service.py:118`).
4. If this block is not itself a boundary, it inherits that state unchanged (`feature_service.py:120 – 121`).
5. If it is a boundary, run `_calculate_new_state` to compute the fresh state, and cache it on this block (`feature_service.py:123 – 135`).

The recursion walks boundary-to-boundary, not block-to-block, and caches as it goes, so after the first pass each boundary's state is a stored lookup. There is one careful detail at step 5: when a boundary block transitions a feature into `MUST_SIGNAL` , the service notifies the bit-signalling service so this node starts voting yes automatically (`feature_service.py:129 – 131`) — and it caches the new state without persisting it, because a brand-new boundary block may still be unverified and its metadata not yet safe to write (`feature_service.py:133 – 135`). The previous boundary, already verified, is the one that gets saved.

38.4.7 Gating behaviour — two real use-sites

A feature is worthless unless something changes when it's active. Two verifiers do exactly that.

Enforcing `MUST_SIGNAL` . The block verifier (Ch. 31) refuses any block that fails to signal a mandatory feature:

```
def verify_mandatory_signaling(self, block: Block) -> None:
    signaling_state = self._feature_service.is_signaling_mandatory_features(block)
    match signaling_state:
        case BlockIsSignaling():
            return
        case BlockIsMissingSignal(feature):
            raise BlockMustSignalError(
                f"Block must signal support for feature '{feature.value}' during MUST_SIGNAL phase"
            )
```

`block_verifier.py:93`

`is_signaling_mandatory_features` (`feature_service.py:69`) checks, for every feature currently in `MUST_SIGNAL` , whether enough blocks remain in the window for the threshold to still be reachable — and if a block's omission would make it unreachable, that block is rejected. This is the teeth behind a forced upgrade: during the `MUST_SIGNAL` window, you cannot mine a valid block without voting yes.

Branching on an active feature. The merge-mined block verifier picks a limit based on whether a feature is live:

```
is_feature_active = self._feature_service.is_feature_active(  
    vertex=block,  
    feature=Feature.INCREASE_MAX_MERKLE_PATH_LENGTH,  
)  
max_merkle_path_length = (  
    self._settings.NEW_MAX_MERKLE_PATH_LENGTH if is_feature_active  
    else self._settings.OLD_MAX_MERKLE_PATH_LENGTH  
)
```

`merge_mined_block_verifier.py:33`

This is the pattern every gated rule follows: ask `is_feature_active` , then apply the new rule or the old one. Before activation height the old limit holds for everyone; after it, the new one — and because every node computes activation from the same shared schedule and the same on-chain votes, every node flips at the same block. No fork.

A convenience wrapper, the `Features` dataclass (`utils.py`), bundles several of these answers into one frozen snapshot for a vertex (`utils.from_vertex`), and folds in a wrinkle: some features can be hard-forced `ENABLED` / `DISABLED` by a plain settings flag instead of going through activation (`utils.py` , `_is_feature_active` , matching on `FeatureSetting`). That is the escape hatch for features that don't need a network vote on a given profile.

38.4.8 BitSignalingService — the write side (this node's ballot)

Everything so far was the read side: given the chain, what's active? The `BitSignalingService` (`bit_signaling_service.py:29`) is the write side: when this node mines a block, which bits does it set?

It is constructed with two operator-provided sets, `support_features` and `not_support_features` (`bit_signaling_service.py:46 - 47`) — the features the operator has explicitly chosen to vote yes or no on, typically via command-line flags. It refuses a feature listed in both (`bit_signaling_service.py:184`).

Its core method assembles the ballot:

```
def generate_signal_bits(self, *, block: Block, log: bool = False) -> int:
    feature_signals = self._calculate_feature_signals(block=block, log=log)
    signal_bits = 0
    for feature, (criteria, enable_bit) in feature_signals.items():
        signal_bits |= int(enable_bit) << criteria.bit
    return signal_bits
```

bit_signaling_service.py:74

It only considers features currently in a signalling state for the given block (bit_signaling_service.py:164, _get_signaling_features, filtering on get_signaling_states()). For each, it decides yes/no (bit_signaling_service.py:106 - 110):

```
enable_bit = (default_enable_bit or support) and not not_support
```

Read it plainly: vote yes if the feature defaults to yes or the operator opted in — unless the operator explicitly opted out. Then generate_signal_bits ORs each yes into the right bit position and returns the byte. The manager stamps that byte onto every block template it produces:

```
signal_bits=self._bit_signaling_service.generate_signal_bits(block=parent_block)
```

manager.py:766

So the loop closes: this node's vote goes onto the blocks it mines → those blocks land in the chain → other nodes' feature_activation_bit_counts tally them → the state machine reads the tally at the next boundary. One more hook: when the read side detects a feature entering MUST_SIGNAL, it calls on_must_signal (bit_signaling_service.py:140), which force-adds the feature to this node's support set — so a node automatically complies with mandatory signalling without operator action.

38.4.9 A small worked example

Make it concrete. Suppose a feature `OPCODES_V2` uses **bit 1**, with `start_height = 200`, `timeout_height = 600`, `threshold = 70`, and an evaluation interval of 100 blocks (toy numbers; mainnet's interval is 20,160). Walk the boundaries:

Window	Boundary block height	Vote count for bit 1 in the window that just ended	State at boundary (after calc)	Why
... → 100	100	(irrelevant)	DEFINED	start_height 200 not reached
100 → 200	200	(irrelevant)	STARTED	height >= start_height
200 → 300	300	55 of 100 set bit 1	STARTED	55 < threshold 70 → keep
300 → 400	400	82 of 100 set bit 1	LOCKED_IN	82 >= 70, before timeout
400 → 500	500	(irrelevant)	ACTIVE	LOCKED_IN + min_activation

From height 400 onward, `is_feature_active(vertex=block_at_height>=500, feature=OPCODES_V2)` returns `True`, and the merge-of opcode rules flip. Every node on the network computes this same table from the same blocks, so they all flip at height 500.

Now the failure path. Keep everything but say the votes never climb: each window lands at 40 of 100. At the boundary of height 600 (`timeout_height`), `previous_state == STARTED`, `height >= timeout_height`, and `lock_in_on_timeout` is `False` → the feature returns **FAILED** (`feature_service.py:170 – 171`) and is never enforced. The proposal died without splitting the chain — which is the whole point.

If instead `lock_in_on_timeout` were `True` (a mandatory feature), the window ending at height 500 would see `height + interval (600) >= timeout (600)` and move to **MUST_SIGNAL** (`feature_service.py:183 – 184`); during that 500→600 window every block would be required to set bit 1 (`block_verifier.py:93`); at the 600 boundary it would move unconditionally to **LOCKED_IN** (`feature_service.py:188 – 192`), then **ACTIVE**.

38.5 How it plugs into the lifecycle

Tracing the wiring across earlier chapters:

- **Build (Ch. 24).** The builder constructs both services and injects them into the `HathorManager`: `feature_service` and `bit_signaling_service` are constructor parameters (`manager.py:110`, `:117`), stored as `self._bit_signaling_service` and `self.feature_service` (`manager.py:197`, `:207`). `BitSignalingService.__init__` also back-links itself onto the feature service (`bit_signaling_service.py:59`) so the read side can call `on_must_signal`.
- **Start (Ch. 29).** During manager start-up, `self._bit_signaling_service.start()` runs (`manager.py:451`). It validates that the persisted feature settings match the current ones (`bit_signaling_service.py:66`, via the storage's `validate_settings`), reads the best block, and logs which features this node is signalling — warning if the operator asked to signal a feature that isn't in a signalling window (`bit_signaling_service.py:190`). The storage layer (`feature_activation_storage.py`) exists so a node cannot silently change a network's feature schedule between restarts without noticing.
- **Mine (Ch. 37).** Every block template the manager produces carries this node's vote, stamped at `manager.py:766`.
- **Read blocks (Ch. 25).** Votes ride in `signal_bits` (`base_transaction.py:168`) and are tallied into `feature_activation_bit_counts` when each block's static metadata is built (`static_metadata.py:132`).

- **Gate rules (Ch. 31).** Verification consults the feature service — `verify_mandatory_signaling` (`block_verifier.py:93`) and the merge-mined merkle-path limit (`merge_mined_block_verifier.py:33`) are the two live examples; future consensus and verification rule changes follow the same `is_feature_active` pattern.
- **Expose (Ch. 36).** `resources/feature.py` serves the current per-feature state over HTTP, so an operator can see exactly where each feature stands.

38.6 Recap

State	Meaning	Leaves to...	On condition (at a window boundary)
DEFINED	In the catalogue; not yet started	STARTED	<code>height >= start_height</code>
STARTED	Voting open; tallying each window	LOCKED_IN	<code>count >= threshold</code> and before <code>timeout_height</code>
		MUST_SIGNAL	nearing timeout and <code>lock_in_on_timeout</code>
		FAILED	reached <code>timeout_height</code> and not <code>lock_in_on_timeout</code>
MUST_SIGNAL	One window; signalling compulsory	LOCKED_IN	always, after exactly one interval
LOCKED_IN	Will activate; grace period	ACTIVE	<code>height >= minimum_activation_height</code>
ACTIVE	Live — new rule enforced	—	terminal
FAILED	Never activates	—	terminal

Feature activation is how Hather changes its own consensus rules without a flag-day: a proposed change is scheduled to a block height, gated on a high threshold of miner votes gathered over fixed windows, and switched on for everyone at once only after the network has demonstrably signalled readiness — or cleanly abandoned if it hasn't. The code splits along a clean seam: `FeatureService` reads the chain to answer "is X active here?" (the question verifiers gate on), and `BitSignalingService` writes this node's votes onto the blocks it mines. The state machine in `_calculate_new_state` is the whole

protocol in one function, and the running `feature_activation_bit_counts` on each block is what makes counting votes cheap. The next chapter opens the single largest subsystem in the node — **nano-contracts** (Ch. 39) — which is itself one of the features gated behind this very mechanism (`Feature.NANO_CONTRACTS`).

1. A fork (specifically a consensus fork or chain split) is when nodes stop agreeing on which blocks are valid, so the single shared ledger splits into two incompatible histories. A coin's state can differ between the two sides. Feature activation exists to prevent forks caused by rule changes; this is distinct from a temporary fork, the brief, self-healing disagreement when two miners find a block at nearly the same time (resolved by consensus, Ch. 32). ↩
2. **BIP9** ("Bitcoin Improvement Proposal 9") is the Bitcoin scheme for "versionbits" soft-fork activation: miners signal readiness for a change by setting bits in a block's version field, and the change activates once a threshold of blocks in a retarget window signal support, within a defined timeout. Hathor's feature activation is a close adaptation of it, with the addition of the `MUST_SIGNAL` mandatory-signalling phase. ↩
3. A `StrEnum` (Python 3.11+) is an enumeration whose members are `str` values. `Feature.NANO_CONTRACTS == 'NANO_CONTRACTS'` is `True`, and `Feature.NANO_CONTRACTS.value` is the string. This lets the settings file refer to features by plain string names while the code uses the type-safe enum. ↩
4. Static metadata is per-vertex data the node computes once and then never changes — as opposed to mutable metadata that gets updated as the ledger evolves (the split is covered in Ch. 25). A block's height and its `feature_activation_bit_counts` are static: once a block is fixed in the chain, both are fixed too, so they can be computed once and cached. ↩